

Auto-Tables: Relationalize Tables without Using Examples

Peng Li*
Georgia Tech
Atlanta, GA
pengli@gatech.edu

Yeye He
Microsoft Research
Redmond, WA
yeyehe@microsoft.com

Cong Yan
Microsoft Research
Redmond, WA
coyan@microsoft.com

Yue Wang
Microsoft Research
Redmond, WA
wanyue@microsoft.com

Surajit Chaudhuri
Microsoft Research
Redmond, WA
surajitc@microsoft.com

ABSTRACT

Relational tables, where each row corresponds to an entity and each column corresponds to an attribute, have been the standard for tables in relational databases. However, such a standard cannot be taken for granted when dealing with tables "in the wild". Our survey of real spreadsheet-tables and web-tables shows that over 30% of such tables do not conform to the relational standard, for which complex table-restructuring transformations are needed before these tables can be queried easily using SQL-based tools. Unfortunately, the required transformations are non-trivial to program, which has become a substantial pain point for technical and non-technical users alike, as evidenced by large numbers of forum questions in places like StackOverflow and Excel/Tableau forums.

We develop an Auto-Tables system that can automatically synthesize pipelines with multi-step transformations (in Python or other languages), to transform non-relational tables into standard relational forms for downstream analytics, obviating the need for users to manually program transformations. We compile an extensive benchmark for this new task, with 244 real test cases collected from user spreadsheets and online forums. Our evaluation suggests that Auto-Tables can successfully synthesize transformations for over 70% of test cases at interactive speeds, without requiring any input from users, making this an effective tool for users to prepare data for downstream analytics.

1. INTRODUCTION

Modern data analytics like SQL and BI are predicated on a standard format of relational tables, where each row corresponds to a distinct "entity", and each column corresponds to an "attribute" for the entities that contains homogeneous data-values. While such tables are de-facto standards in relational databases, to the extent that we as database people may take them for granted, we would like the highlight that a significant fraction of tables "in the wild" actually fail to conform to such standards, making it considerably more challenging to query them using SQL-based tools.

Non-relational tables are common, but hard to query. Real tables in the wild, such as spreadsheet-tables or web-tables, can often be "non-relational" and hard to query, unlike tables that we expect to find in relational databases. We randomly sampled hundreds of user spreadsheets (in Excel), and web tables (from Wikipedia), and found around 30-50% tables to have such issues. Figure 1 and Figure 2 show real samples taken from spreadsheets and the web, respectively, to demonstrate these common issues. (We emphasize that the problem is prevalent at a very large scale, since there are millions of tables like these in spreadsheets and on the web.)

Take Figure 1(a) for example. The table on the left is not a standard relational table, because each column marked in green contains sales numbers for only a single day ("`19-Oct`", "`20-Oct`", etc.), making these column values highly homogeneous in the horizontal direction (while in typical relational tables, we expect values in columns to be homogeneous in the vertical direction). Although this specific table format makes it easy for humans to eyeball changes day-over-day by reading horizontally, it is unfortunately hard to analyze using SQL. Imagine that one needs to compute the 14-day average of sales, starting from "`20-Oct`" – for this table, one has to write: `SELECT SUM("20-Oct", "21-Oct", "22-Oct", ...) FROM T`, across 14 different columns, which is long and unwieldy to write. Now imagine we need 14-day moving averages with every day in October as the starting date – the resulting SQL is highly repetitive and hard to manage.

In contrast, consider a transformed version of this table, shown on the right of Figure 1(a). Here the homogeneous columns in the original table (marked in green) are transformed into only two new columns: "`Date`" and "`Units Sold`", using a transformation operator called "`stack`" (listed in the first row of Table 1). This transformed table contains the same information as the original table, but is much easier to query – e.g., the same 14-day moving average can be computed using a succinct range-predicate on the '`Date`" column, where the starting date "`20-Oct`" is a literal parameter that can be easily changed into other values.

There are many such spreadsheet tables that require different kinds of transformations before they are ready for SQL-based analysis. Figure 1(b) shows another example, where every 3 columns form a repeating group, representing "`Revenue/Units Sold/Margin`" for a different year (marked in red/green/blue in the figure). Tables with these repeat-

*Part of work done while at Microsoft.

Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment. This is a minor revision of the paper entitled "Auto-Tables: Synthesizing Multi-Step Transformations to Relationalize Tables without Using Examples", published in PVLDB, Vol. 16, No. 11, 2150-8097. DOI: https://doi.org/10.14778/3611479.3611534

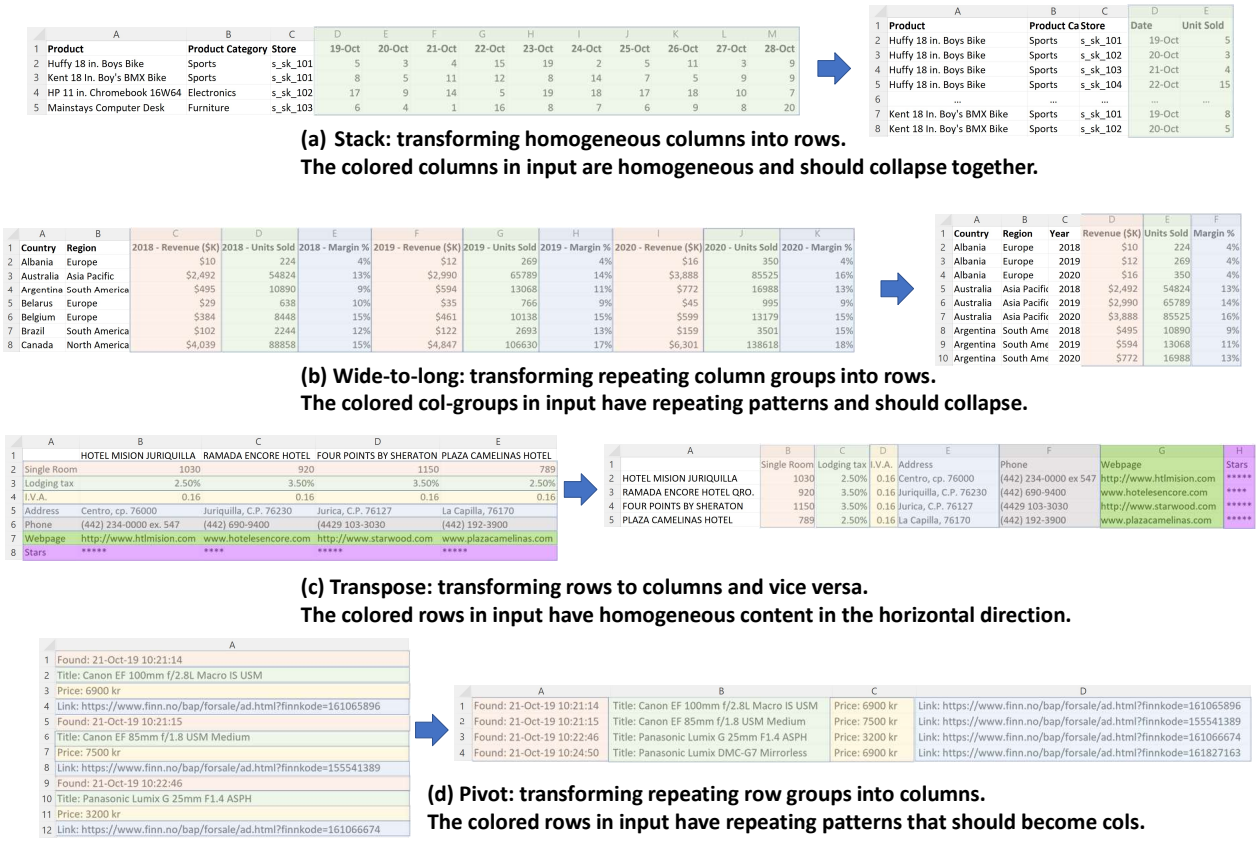


Figure 1: Example input/output tables for 4 operators in AUTO-TABLES: (a) Stack, (b) Wide-to-long, (c) Transpose, (d) Pivot. The input-tables (on the left) are not relational and hard to query, which need to be transformed to produce corresponding output-tables (on the right) that are relational and easy to query. Observe that the color-coded, repeating row/column-groups are "visual" in nature, motivating a CNN-like architecture like used in computer vision for object-detection.

ing column-groups are also hard to query, just like Figure 1(a), but in this case the required transformation operator is called "`wide-to-long`" (second row of Table 1).

Figure 1(c) shows yet another example, where each hotel corresponds to a column (whose names are in row-1), and each "attribute" of these hotels corresponds to a row. Note that in this case values in the same rows are homogeneous (marked in different colors), unlike relational tables where values in the same columns are homogeneous. A transformation called "`transpose`" is required in this case (listed in the third row of Table 1), to make the resulting table, shown on the right of the figure, easy to query – for instance, a query to sum up the total number of hotel rooms is hard to write on the original table, but can be easily achieved using a simple SUM query on the "`Single Room`" column in the transformed table.

Figure 1(d) shows another example where columns are represented as rows in the table on the left. This is similar to Figure 1(c), except that the rows in this case are "repeating" in groups, thus requiring a different transformation operator called "`pivot`" (listed in the fourth row of Table 1) as opposed to "`transpose`". The resulting table is shown on the right, which becomes easy to query.

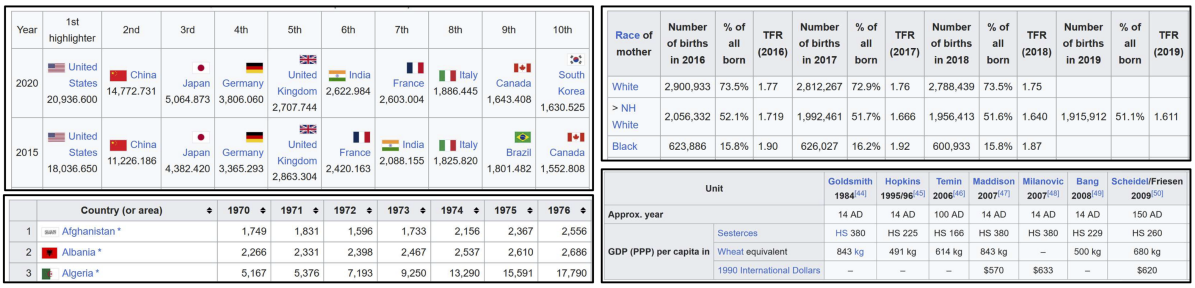
While the examples so far are all taken from spreadsheets, we note that similar structural issues are also widespread in Web tables. Figure 2 shows real examples from Wikipedia, which share similar characteristics as the spreadsheet tables in Figure 1, which all require transformations before these tables can be queried effectively.

Non-relational tables are hard to "relationalize". We mentioned that the example tables in Figure 1 and Figure 2 require different transformation operators. Table 1 shows 8 such transformation operators commonly needed to relationalize tables (where the first 4 operators correspond to the examples we see in Figure 1).

The first column of Table 1 shows the name of the "logical operator", which may be instantiated differently in different languages (e.g., in Python or R), with different names and syntax. The second column of the table shows the equivalent Pandas operator in Python [11], which is a popular API for manipulating tables among developers and data scientists, that readers may be familiar with.

While the operations listed in Table 1 already exist in languages such as R and Python, they are not easy for users to invoke correctly, because users need to:

1. Visually identify different structural issues in an input table that make it hard to query (e.g., repeating row-/column-groups shown in Fig. 1(a-d)), which is not obvious to non-expert users;
2. Map the visual pattern identified from the input table, to a corresponding operator in Table 1 that can handle such issues. This is hard for users not familiar with the exact terminologies to describe these transformations (e.g., `pivot` vs. `stack`);



Year	1st highlighter	2nd	3rd	4th	5th	6th	7th	8th	9th	10th
2020	United States 20,936.600	China 14,772.731	Japan 5,064.873	Germany 3,806.060	United Kingdom 2,707.744	India 2,622.984	France 2,603.004	Italy 1,886.445	Canada 1,643.408	South Korea 1,630.525
2015	United States 18,036.650	China 11,226.186	Japan 4,382.420	Germany 3,365.293	United Kingdom 2,863.304	France 2,420.163	India 2,088.155	Italy 1,825.820	Brazil 1,801.482	Canada 1,552.808

Race of mother	Number of births in 2016	% of all born	TFR (2016)	Number of births in 2017	% of all born	TFR (2017)	Number of births in 2018	% of all born	TFR (2018)	Number of births in 2019	% of all born	TFR (2019)
White	2,900,933	73.5%	1.77	2,812,267	72.9%	1.76	2,788,439	73.5%	1.75			
> NH White	2,056,332	52.1%	1.719	1,992,461	51.7%	1.666	1,956,413	51.6%	1.640	1,915,912	51.1%	1.611
Black	623,886	15.8%	1.90	626,027	16.2%	1.92	600,933	15.8%	1.87			

	Country (or area)	1970	1971	1972	1973	1974	1975	1976
1	Afghanistan *	1,749	1,831	1,596	1,733	2,156	2,367	2,556
2	Albania *	2,266	2,331	2,398	2,467	2,537	2,610	2,686
3	Algeria *	5,167	5,376	7,193	9,250	13,290	15,591	17,790

Unit		Goldsmith 1984[44]	Hopkins 1995/96[45]	Temin 2006[46]	Maddison 2007[47]	Milanovic 2007[48]	Bang 2008[49]	Scheidel/Friesen 2009[50]
Approx. year		14 AD	14 AD	100 AD	14 AD	14 AD	14 AD	150 AD
GDP (PPP) per capita in	Sesterces	HS 380	HS 225	HS 166	HS 380	HS 380	HS 229	HS 260
	Wheat equivalent	843 kg	491 kg	614 kg	843 kg	–	500 kg	680 kg
	1990 International Dollars	–	–	–	$570	$633	–	$620

Figure 2: Real Web tables from Wikipedia that are also non-relational, similar to the spreadsheet tables shown in Figure 1.

Table 1: Auto-Tables DSL: table-restructuring operators and their parameters to "relationalize" tables. These operators are common and exist in many different languages, like Python Pandas and R, sometimes under different names.

DSL operator	Python Pandas equivalent	Operator parameters	Description (example in parenthesis)
stack	melt [14]	start_idx, end_idx	collapse homogeneous cols into rows (Fig. 1a)
wide-to-long	wide_to_long [18]	start_idx, end_idx, delim	collapse repeating col-groups into rows (Fig. 1b)
transpose	transpose [17]	-	convert rows to columns and vice versa (Fig. 1c)
pivot	pivot [15]	repeat_frequency	pivot repeating row-groups into cols (Fig. 1d)
explode	explode [12]	column_idx, delim	convert composite cells into atomic values
ffill	ffill [13]	start_idx, end_idx	fill structurally empty cells in tables
subtitles	copy, ffill, del	column_idx, row_filter	convert table subtitles into a column
none	-	-	no-op, the input table is already relational

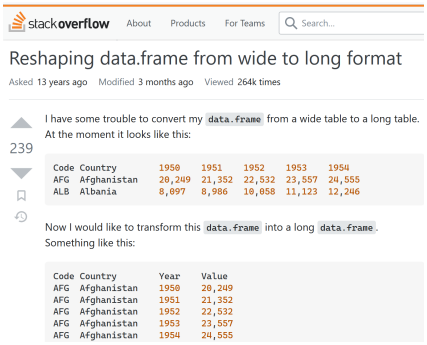


Figure 3: Example user question from StackOverflow, on how to restructure tables. Questions like this are common not only among technical users, but also non-technical users, as similar questions are commonly found on forums for Excel, Power-BI, and Tableau users too [6, 7, 8, 9].

3. Parameterize the chosen operator appropriately (e.g., which columns to collapse, what is the repeating frequency, etc.). This is again hard, as even developers need to consult API documentations that can be long and complex.
4. Certain input tables require more than one transformation step, for which users need to repeat steps (1)-(3).

Completing these steps is a tall order even for technical users, as evidenced by a large number of such questions on forums like StackOverflow. Figure 3 shows one example (popular with many up-votes) – the developer provides example input/output tables to demonstrate the desired transformation, in order to ask what operators should be used.

If technical users like developers find it hard to restructure their tables as these StackOverflow questions would show, it comes as no surprise that non-technical enterprise users, who often deal with tables in spreadsheets, would find the task even more challenging. We find a large number of similar questions on Excel and Tableau forums (e.g., [6, 7, 8, 9]), where users complain that without the required transformations it is hard to analyze data using SQL-based or Excel-based tools (e.g., [2, 3, 4, 5]). The prevalence of these questions confirms table-restructuring as a common pain point for both technical and non-technical users.

Auto-Tables: synthesize transformations without examples. In this work, we propose a new paradigm to automatically synthesize table-restructuring steps to relationalize tables, using the operators in Table 1, *without requiring users to provide examples*. Our key intuition of why we can do away with examples in our task, lies in the observation that given an input table, the logical steps required to relationalize it are *almost always unique*, as the examples in Figure 1 would all show. This is because the transformations required in our task only "restructure" tables, that do not actually "alter" the table content, which is unlike prior work that focuses on *row-to-row transformations* (e.g., TDE [31] and FlashFill [29]), or SQL-by-example (e.g. [22, 50]), where the output is "altered" that can produce many possible outcomes, which would require users to provide input/output examples to demonstrate the desired outcome.

For our task, we believe it is actually important *not* to ask users to provide examples, because in the context of table-to-table transformations like in our case, asking users to provide examples would mean users have to specify *an output table*, which is a substantial amount of typing effort, making it cumbersome to use.

As humans, we can "visually" recognize rows/columns patterns (e.g., homogeneous value groups, as color-coded vertically and horizontally in Figure 1), to correctly predict which operator to use. The question we ask in this paper, is whether an algorithm can "learn" to recognize such patterns by scanning the input tables alone, to predict suitable transformations, in a manner that is analogous to how computer-vision algorithms would scan a picture to identify common but more complex objects like dogs and cats.

In computer vision, in order to pick up subtle clues from

CM-Explorer: Dissecting Data Ingestion Problems

Niels Bylois
UHasselt
Data Science Institute, ACSL
Belgium
niels.bylois@uhasselt.be

Frank Neven
UHasselt
Data Science Institute, ACSL
Belgium
frank.neven@uhasselt.be

Stijn Vansummeren
UHasselt
Data Science Institute, ACSL
Belgium
stijn.vansummeren@uhasselt.be

ABSTRACT

Data ingestion validation, the task of certifying the quality of continuously collected data, is crucial to ensure trustworthiness of analytics insights. A widely used approach for validating data quality is to specify, either manually or automatically, so-called data unit tests that check whether data quality metrics lie within expected bounds. We employ conditional unit tests based on conditional metrics (CMs) that compute data quality signals over specific parts of the ingestion data and therefore allow for a fine-grained detection of errors. A violated conditional unit test specifies a set of erroneous tuples in a natural way: the subrelation that its CM refers to. Unfortunately, the downside of their fine-grained nature is that violating unit tests are often correlated: a single error in an ingestion batch may cause multiple tests (each referring to different parts of the batch) to fail. The key challenge is therefore to untangle this correlation and filter out the most relevant violated conditional unit tests, i.e., tests that identify a core set of erroneous tuples *and* act as an explanation for the errors. We present CM-Explorer, a system that supports data stewards in quickly finding the most relevant violated conditional unit tests. The system consists of three components: (1) a graph explorer for visualizing the correlation structure of the violated unit tests; (2) a relation explorer for browsing the tuples selected by conditional unit tests; and, (3) a history explorer to get insight why conditional unit tests are violated. In this paper, we discuss these components and present the different scenarios that we make available for the demonstration.

PVLDB Reference Format:
Niels Bylois, Frank Neven, and Stijn Vansummeren. CM-Explorer: Dissecting Data Ingestion Problems. PVLDB, 16(12): 3958 - 3961, 2023.
doi:10.14778/3611540.3611595

1 INTRODUCTION

Modern data analytics pipelines continuously collect and ingest new data. Validating the quality of collected data at ingestion time is crucial in such pipelines, for a number of reasons. First, and foremost, the quality of the derived insights, and the decisions driven by them, depend directly on the quality of the collected data [8]. Second, as more and more of the data analysis process is automated, small errors in source data risk propagating to later data consumers (such as machine learning models), which may themselves act as data sources in other processing pipelines—thereby potentially magnifying the error [3]. Finally, data errors may even cause data processing pipelines to crash (e.g., because of null pointer exceptions due to missing data). Machine learning platforms are therefore including explicit data validation components into their pipelines [1, 2, 5].

In recognition of the importance of data quality validation in modern analytics pipelines, several tools have been proposed to aid in automatic validation [3, 7, 8]. Broadly speaking, these tools allow specification, either manually or automatically, of so-called *data unit tests*. When a new batch of data is to be ingested, the registered tests are executed to gauge the batch's quality where failing tests highlight data quality problems. The tests themselves entail computing certain metrics on the data batch (e.g., the minimum or maximum value appearing in a numerical column or the number of distinct elements appearing in a column) and checking that these fall within an expected range.

Unfortunately, these tools suffer from two limitations. First, the data unit tests that they support are based on *global* metrics: metrics that are computed on the entire data batch, or on an entire column in the batch. As the following example shows, they hence only provide *coarse-grained* signals of data quality and are unable to detect *fine-grained* errors, i.e., errors that occur only in a specific (potentially small) part of the batch. Batches with fine-grained errors hence go unnoticed. Second, even when a data unit test signals that a batch has a data quality problem, it does not provide a principled method to identify the part of the batch that is responsible for a test's failure. As such, either the entire batch must be discarded or a human expert must manually identify and subsequently remove or correct the erroneous tuples—which is time-consuming and labor-intensive.

Example 1.1. A public railway company has equipped all of its trains with measurement sensors that record the train's arrival and departure time at each train station. By comparing these times with the time schedule, the train's software computes the corresponding delays (if any). At the end of each day, the measurements of all trains are collected, and ingested in the railway company's data lake. The delays are used to identify hotspot routes, as well as computation of service quality indicators to the government. Train 5437 runs daily from Hasselt (Belgium) to Blankenberge (Belgium). This route is notorious for the delays that it incurs when it passes through the busy Brussels railway stations. As such, train 5437 normally reports non-zero delay. Due to a hardware malfunction on March 15, however, it consistently reports zero delay for this train.

The metrics used by state-of-the-art tools are unable to detect this error. Indeed: because zero delay is *not* an uncommon value when considering the entire ingestion batch (some trains run on

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 12 ISSN 2150-8097.
doi:10.14778/3611540.3611595

time), metrics such as min(delay), max(delay), and avg(delay) will not consider zero delay as an anomaly.

It is important to observe that, even if one of the global metrics signals a data quality problem, it is not clear which train or set of trains in the batch cause the problem. For instance, if the unit tests based on min(delay) or max(delay) signal an anomaly then of course we can easily identify the erroneous trains: simply compute the trains whose delay value is below (or above) the expected minimum (resp. maximum) value. However, if a unit test based on avg(delay) signals a problem, then it is unclear how to identify the trains that caused the delay to deviate. □

We have developed an approach for data quality validation that is not only capable of detecting the *fine-grained errors* illustrated above, but also helps in *identifying* responsible erroneous tuples [4]. Our methodology focuses on the setting where a data pipeline regularly ingests batches of external data. As the main technical vehicle underlying our methodology, we introduce *conditional metrics* (CM for short). In contrast to a global metric, a conditional metric only computes its value on a specified subset of tuples in the ingestion batch. A CM m is of the following form

$$m := \mu(Y \mid X = x) \qquad (\dagger)$$

where μ is a metric like MIN, MAX, SUM, MAXDIGITS, ...; Y and X are attributes of the relation under consideration and x is a domain value for X. We refer to (X, x) as the entity in m. The semantics is as follows: m computes the metric μ over the column Y for the subrelation consisting of all tuples where X equals the value x. In the railway example above, for instance, the conditional metric

$$\textsc{avg}(\mathit{Delay} \mid \mathit{Train} = 5437)$$

computes the average delay of train 5437 in the batch, as opposed to the global metric AVG(*Delay*) which computes the average delay of all trains. A conditional unit test then is simply a CM with an associated function that returns true for admissible values and false otherwise. Because of their ability to calculate values for specific entities (e.g., *Train* = 5437), conditional data unit tests may hence detect data quality issues at a finer level of granularity than data unit tests based on global metrics.

Our methodology consists of two phases: (i) a unit test discovery phase and (ii) a monitoring and error identification phase, as schematically illustrated in Figure 1. In the *unit test discovery phase*, we are given a sequence $\overline{R}$ of previously ingested batches and our objective is to automatically derive a set Θ of CM-based data unit tests from $\overline{R}$ such that a yet-to-be-ingested batch B can be considered to be of acceptable quality if it passes all tests in Θ. Specifically, like [7], we assume that the batches in $\overline{R}$ are themselves of acceptable data quality—which is reasonable because they have already been ingested successfully. Under this assumption, we may derive CM-based data unit tests from $\overline{R}$ by considering the set of all possible CMs, and deriving, for each such CM m, a classifier that allows to distinguish expected values of m from anomalies. If m produces an anomaly on a yet-to-be-ingested batch B, then B will be flagged as having a quality issue. To derive the classifier based on the values of m observed in $\overline{R}$, in principle any anomaly detection method [6] may be used. For illustration purposes, Figure 1 mentions some simple anomaly detection methods that produce classifiers that

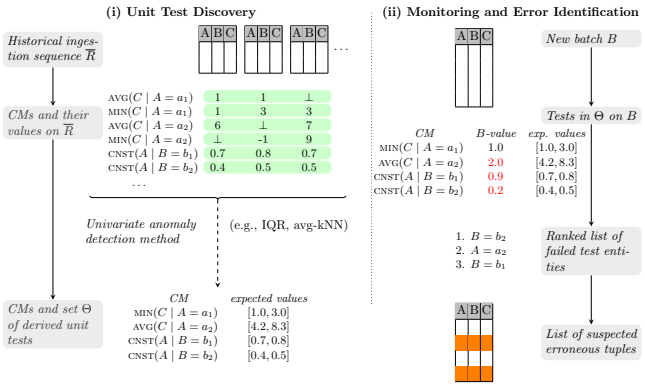


Figure 1: Overview of methodology: (i) *discovery phase* **to derive data unit tests based on conditional metrics; (ii)** *monitoring and error identification phase* **where each new batch is validated and erroneous tuples are identified.**

can be summarized as an expected range of values, where values outside the range are anomalies. Whatever method is chosen, we require that the derived classifier is consistent with our assumption that $\overline{R}$ is of reasonable quality, which implies that the values of m observed in $\overline{R}$ are without significant anomalies.

In the *monitoring and error identification phase*, we take the set Θ of data unit tests derived in the discovery phase, and use this to validate each new ingestion batch B. If all tests in Θ succeed on B then B is deemed to be of acceptable quality. When at least one test in Θ rejects B then our objective is to identify the tuples in B with suspected errors. Every conditional unit test specifies a set of erroneous tuples in a natural way: the subrelation of B that its CM refers to. In our railway example, if a unit test with CM AVG(*Delay* | *Train* = 5437) hence fails, all tuples with *Train* = 5437 could be flagged. Simply flagging *all* the tuples of violated unit tests selects too much, however (i.e., it results in high recall but very low precision). The reason is that violating tests are often correlated: a single error in B may cause multiple tests (each selecting different subrelations) to fail. The key challenge in the monitoring phase, therefore, lies in *ranking* the violated unit tests according to relevance, and from this ranked list of tests *filter* a list of suspected erroneous tuples for further inspection. We have developed multiple metrics to rank and filter violated unit tests in [4].

We illustrate the challenges in analyzing violated unit tests by means of an example. For this, we introduce the bipartite entity-tuple graph as a means to represent the correlation structure of the violated unit tests. This graph consists of two sets of nodes: all entities mentioned in at least one of the violated unit tests and every tuple that is selected by at least one of them. Furthermore, there is an edge from an entity to a tuple when that tuple is selected by the entity. Recall the scenario outlined in Example 1.1, where due to a hardware malfunction train 5437 consistently reports zero delay. Consider the to-be-ingested batch B shown in Figure 2 as well as the failed unit tests ϕ_1, ϕ_2, and ϕ_3 listed there. The entity-tuple graphs is displayed as well. We note that for the sake of this example, we have added an extra attribute *Kind* that lists the kind of service that the train offers. Unit test ϕ_1 fails because the average delay of train

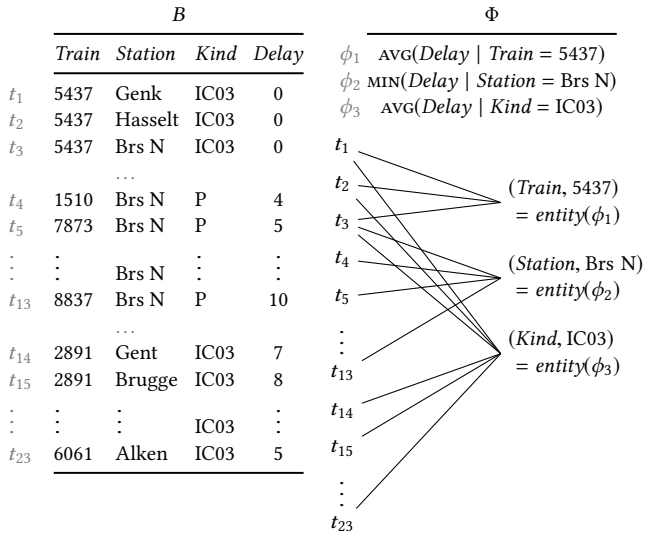


Figure 2: Illustration of a to-be-ingested batch B, failed unit tests Φ, and the corresponding entity-tuple graph.

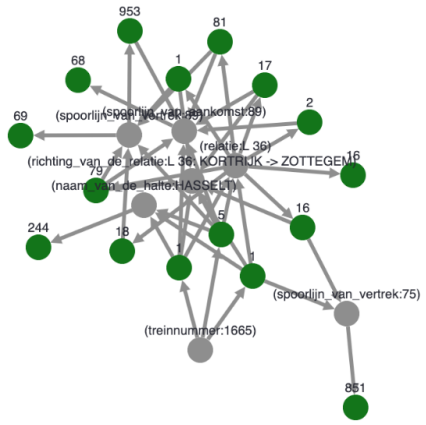


Figure 3: Entity-tuple graph explorer example for a single component for all *flagged* entities.

5437 is now zero, which is unexpected given the historical sequence. However, in this example, the zero delay of train 5437 also causes the minimum delay in station Brussels North (Brs N) to become zero, which is also unexpected, causing ϕ_2 to fail. Similarly, it also causes the average delay of route kind IC03 to become unexpected, failing ϕ_3. The root cause here is the zero delay of train 5437, and ideally we would hence like to filter out ϕ_1 that selects only the tuples t_1–t_3 of train 5437 as suspected erroneous tuples.

We present CM-Explorer, an interactive system that focuses on the monitoring and error identification phase of the methodology described above, and that allows to interactively investigate the observed data quality problems. Using CM-Explorer, a data steward may easily inspect the correlation structure of the violated unit tests via a graph-based representation to identify the most relevant ones for selecting a set of erroneous tuples and serving as an explanation for them. We next describe the components of CM-Explorer that support this (in Section 2), and discuss concrete demonstration scenarios in Section 3.

2 OVERVIEW OF CM-EXPLORER

The CM-Explorer consists of three components: the graph explorer, the CM history explorer, and the relation explorer. The workflow supported by CM-Explorer is as follows. A data steward may first utilise the graph explorer to filter the failed conditional unit tests by selecting tests with high scoring entities and removing tests with entities that refer to column names that are not considered to be important. In a second step, the data steward can zoom into individual entities using the history and relational explorer, to further reduce the conditional unit tests until only those of interest remain.

2.1 Graph explorer

An example of a real-world entity-tuple graph, as inspectable in the graph explorer, is displayed in Figure 3. The graph explorer always shows a contracted version of the entity-tuple graph, where tuple nodes are combined into a single node when they are connected to the same set of entity nodes. Contracted tuple nodes are colored green and carry as label the number of tuples that they represent. Entity nodes are colored grey. The graph explorer supports the data steward in analyzing the entity-tuple graph through a number of filters:

- Filtering on score: As explained above, we developed a number of metrics to rank entities and each of those can be used to filter for high scoring entities (and the associated CMs). When a particular measure is chosen, the score of each entity can be displayed.
- Filtering on entity column name: Consider the conditional metric AVG($Delay$ | $Train$ = 5437). Then $Train$ refers to the entity column name and $Delay$ to the column name the aggregate is applied to. The data steward can filter out specific column names that she regards as uninteresting thereby reducing the size of the entity-tuple graph.

To further support interactive exploration, the graph explorer component allows to remove specific entities from the graph, that, for instance, have been inspected by the components described in the following sections, and for which it has been established that they do not refer to erroneous tuples and can be safely disregarded.

2.2 History explorer

When an entity is selected in the graph explorer, the history view shows all the data points from previous batches in blue, with the lower and upper bounds highlighted in red (see Figure 4a for an example). The value on the current batch (that reported an error) is shown in green. When there is no value for a specific batch in the historical data, this is shown as a gap in the graph. The user can hover over a specific data point to inspect the exact value for that batch. The user can hence explore in more detail why specific

Fast and Scalable Data Transfer Across Data Systems

HARALAMPOS GAVRIILIDIS, BIFOLD and Technische Universität Berlin, Germany
KAUSTUBH BEEDKAR, Indian Institute of Technology Delhi, India
MATTHIAS BOEHM, BIFOLD and Technische Universität Berlin, Germany
VOLKER MARKL, BIFOLD, Technische Universität Berlin, and DFKI GmbH, Germany

Fast and scalable data transfer is crucial in today's decentralized data ecosystems and data-driven applications. Example use cases include transferring data from operational systems to consolidated data warehouse environments, or from relational database systems to data lakes for exploratory data analysis or ML model training. Traditional data transfer approaches rely on efficient point-to-point connectors or general middleware with generic intermediate data representations. Physical environments (e.g., on-premise, cloud, or consumer nodes) also have become increasingly heterogeneous. Existing work still struggles to achieve both, fast and scalable data transfer as well as generality in terms of heterogeneous systems and environments. Hence, in this paper, we introduce a holistic data transfer framework. Our XDBC framework splits the data transfer pipeline into logical components and provides a wide variety of physical implementations for these components. This design allows a seamless integration of different systems as well as the automatic optimizations of data transfer configurations according to workload and environment characteristics. Our evaluation shows that XDBC outperforms state-of-the-art generic data transfer tools by up to 5×, while being on par with specialized approaches.

CCS Concepts: • **Information systems** → *Data federation tools*; **Extraction, transformation and loading**; **Data exchange**.

Additional Key Words and Phrases: Data Transfer, Data Loading Optimization, Data System Interoperability

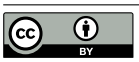
ACM Reference Format:
Haralampos Gavriilidis, Kaustubh Beedkar, Matthias Boehm, and Volker Markl. 2025. Fast and Scalable Data Transfer Across Data Systems. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 157 (June 2025), 28 pages. https://doi.org/10.1145/3725294

1 Introduction

The typical IT landscape of today's organizations comprises various decentralized data ecosystems and many data-driven applications [29, 34]. Over the last decades, there was a trend towards specialized data systems to improve performance and functionality [84, 85]. Apart from processing systems, modern data lakes allow storing raw data in various formats. Since data is distributed across these heterogeneous data representations and systems, efficient data transfer mechanisms are crucial for seamless integration and data analysis.

A Need for Data Transfer: There is a strong need for efficient data transfer because data movement and conversion is often resource-intensive and time-consuming. Accordingly, there is work on eliminating unnecessary data transfer. For example, in-database machine learning aims to conduct training and especially inference inside the DBMS [22, 49, 77] to avoid costly exports from the DBMS.

Authors' Contact Information: Haralampos Gavriilidis, BIFOLD and Technische Universität Berlin, Germany, gavriilidis@tu-berlin.de; Kaustubh Beedkar, Indian Institute of Technology Delhi, India, kbeedkar@cse.iitd.ac.in; Matthias Boehm, BIFOLD and Technische Universität Berlin, Germany, matthias.boehm@tu-berlin.de; Volker Markl, BIFOLD, Technische Universität Berlin, and DFKI GmbH, Germany, volker.markl@tu-berlin.de.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/6-ART157
https://doi.org/10.1145/3725294

Similarly, IoT systems push computation down to the edge to reduce unreliable network transfers [31, 92]. However, there are many use cases where data must be moved, either to leverage specialized tools, to integrate systems, or to consolidate data, e.g., data science applications [90], federated systems [26, 41, 44, 76, 78], extraction-transformation loading (ETL) flows [80], or data replication [56]. Overall, many modern data-driven applications would benefit from efficient data transfer mechanisms.

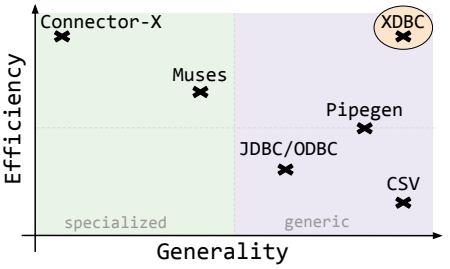


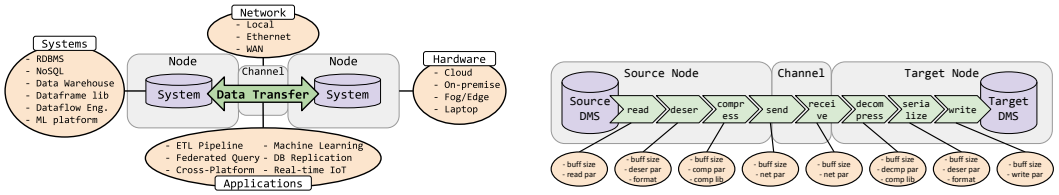
Fig. 1. Data Transfer Approaches.

Data Transfer Challenges: Designing an efficient data transfer mechanism is challenging due to the heterogeneity of data systems, particularly in terms of interfaces, data formats, and varying environment characteristics. Data transfer across heterogeneous systems is commonly realized through custom (i.e., specialized) point-to-point connectors, or generic solutions such as integration middleware or generic JDBC/ODBC drivers [6]. Figure 1 shows this trade-off space of efficiency versus generality. First, specialized connectors provide good performance but lack generality because they are implemented for specific pairs of systems. Second, generic adapters like JDBC/ODBC are primarily designed for lightweight DBMS access and suffer from poor performance [74]. Additionally, data transfer involves multiple steps (e.g., reading, deserializing, compressing, and transmitting), which must be tuned for data, system, and network characteristics. For example, compression may improve the transfer in unreliable consumer-cloud networks, but yield slowdowns in a cloud environment. The complexity of heterogeneous systems and environments renders the optimization of data transfer onerous. Hence, there is a need for a data transfer mechanism that yields robust performance, is easy to use, and general enough for data transfer across heterogeneous systems.

Limitations of Existing Work: Despite work on optimizing ETL [80, 81] and integration flows [28], the literature on efficient data transfer is relatively sparse. Raasveldt et al. analyzed DBMS client protocols and found that JDBC-based protocols are unsuitable for transferring large amounts of data due to their row-based nature and excessive metadata in each protocol buffer [74]. Pipegen generates pipes from existing CSV I/O unit tests in DBMSes, and redirects text I/O to compressed binary communication [48]. Zigzag joins optimize data transfers during joins of data warehouse tables and files in distributed file-system via Bloom filters [86]. Connector-X improves the loading of DBMS tables into pandas dataframes through parallelism and efficient data handling [90]. While these approaches optimize specific aspects of data transfer, there is a lack of a holistic data transfer framework that adapts to different physical environments and supports heterogeneous systems.

Contributions: To address the data transfer challenges and limitations of existing work, we introduce XDBC as a holistic data transfer framework. XDBC offers reader and writer interfaces that can be implemented for arbitrary data systems, eliminating the need for multiple connectors per system pair. XDBC's architecture comprises decoupled components (for read, deserialize, compress, send, receive, decompress, serialize, and write) as well as different physical implementations, which facilitate a flexible configuration and tuning for different environments and data characteristics. In order to free users from the need of manual tuning, XDBC's heuristic optimizer automatically selects effective configurations for given environments. Our detailed technical contributions are:

- **XDBC Architecture:** We describe the architecture and runtime of XDBC in Section 3. The simple yet very effective design allows seamlessly integrating existing data systems as well as tuning the individual data transfer components.



(a) Data Transfer Topologies and Applications.

(b) Data Transfer Anatomy: Configurable Pipeline.

Fig. 2. Data Transfer Across Heterogeneous Systems

- **XDBC Optimizer:** We introduce a novel optimizer for automatically tuning the data transfer configurations in different environments in Section 4. This heuristic optimizer selects parallelization and transfer strategies, and adapts configurations to monitored runtime statistics.
- **Experiments:** Finally, we present the results of an extensive experimental evaluation in Section 5. We compare XDBC with a variety of baselines on real and synthetic datasets as well as different simulated network topologies.

2 Background

In this section, we introduce the necessary background of data transfer, guided by a running example. Data transfer is crucial for many applications, and existing solutions fall short for generic data transfers across heterogeneous systems in different environments.

2.1 Motivation and Running Example

As shown in Figure 2a, data transfer has many different dimensions: different applications run on different data systems and infrastructures (hardware and networking). In the following, we introduce a running example and describe the state-of-the-art data transfer.

Example 1 (Data Science Use Case). *Assume a data-science application, where data practitioners need to load a large dataset from a DBMS, such as PostgreSQL, into a data science framework, such as Python pandas, to pre-process the data, run exploratory data analysis, and train various ML models. The data transfer between the DBMS and pandas may occur in different environments with very different characteristics. First, pandas may run on a local laptop, connected via a slow wide-area-network to the cloud. Second, the data practitioner may run pandas in a remote, cloud-hosted, environment, such as Google Colab, which is better connected to the DBMS.*

Generic Data Transfer: To load the data from the DBMS into pandas, one can utilize generic ODBC-based connectors, such as the widely-used `turbodbc` [12] and `sql.reader` [8]. Such plug-and-play connectors are very practical: users install the drivers and issue queries against the DBMS. The row-based results are then put into a collection of objects such as a pandas dataframe. Most ODBC (and JDBC) drivers only offer limited tuning knobs to optimize the transfer. The most commonly used tuning parameter is the number of rows to fetch in each round-trip to the DBMS. To further optimize performance, users could manually issue multiple queries—that partition the table by primary key—and then assemble the partitions. Existing data loading tools (e.g., Connector-X [90] and Spark JDBC [11]) also follow this approach and generate the partitioning queries automatically. However, the used row format—ideal for row-oriented DBMSes—has shown to be inefficient due to metadata bloat of in-flight buffers and limited compressibility [74].

Specialized Data Transfer: An alternative solution is a custom (i.e., specialized) connector that bypasses the generic JDBC/ODBC driver and establishes a direct connection to the DBMS. This approach allows more fine-grained control and direct conversion from the DBMS-internal



Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes

Simran Arora
Stanford University
simarora@stanford.edu

Brandon Yang
Stanford University
bcyang@stanford.edu*

Sabri Eyuboglu
Stanford University
eyuboglu@stanford.edu*

Avanika Narayan
Stanford University
avanikan@stanford.edu

Andrew Hojel
Stanford University
ahojel@stanford.edu

Immanuel Trummer
Cornell University
itrummer@cornell.edu

Christopher Ré
Stanford University
chrismre@stanford.edu

ABSTRACT

A long standing goal in the data management community is developing systems that input documents and output queryable tables without user effort. Given the sheer variety of potential documents, state-of-the art systems make simplifying assumptions and use domain specific training. In this work, we ask whether we can maintain generality by using the in-context learning abilities of large language models (LLMs). We propose and evaluate Evaporate, a prototype system powered by LLMs. We identify two strategies for implementing this system: prompt the LLM to directly extract values from documents or prompt the LLM to synthesize code that performs the extraction. Our evaluations show a cost-quality tradeoff between these two approaches. Code synthesis is cheap, but far less accurate than directly processing each document with the LLM. To improve quality while maintaining low cost, we propose an extended implementation, Evaporate-Code+, which achieves better quality than direct extraction. Our insight is to generate many candidate functions and ensemble their extractions using weak supervision. Evaporate-Code+ outperforms the state-of-the art systems using a *sublinear* pass over the documents with the LLM. This equates to a 110× reduction in the number of documents the LLM needs to process across our 16 real-world evaluation settings.

PVLDB Reference Format:
Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. PVLDB, 17(2): 92 - 105, 2023.
doi:10.14778/3626292.3626294
PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/HazyResearch/evaporate.

1 INTRODUCTION

Organizations often seek insights trapped in heterogeneous data lakes (*e.g.* the web, corporate data lakes, and electronic health records) [10, 26, 54]. In their raw form, these data sources cannot easily support analytical queries. A long standing goal of the data management community is to develop systems that automatically convert heterogeneous data lakes into queryable, structured tables [12, 15, 47, 66, inter alia.]. In this work, we investigate whether recent large language models can help address this problem.

We study systems that take as **input** heterogeneous documents (*e.g.* HTML webpages, PDFs, text) and **output** a tabular, structured view of the documents. These systems must identify the schema and perform extraction to populate the table.

> Example 1. Medical researchers frequently use data spanning electronic health records (EHR), clinical trials, knowledge sources (e.g. PubMed), and FDA reports to understand and monitor patients and treatments [8]. Consider the large collection of **FDA 510(k)** reviews for premarket medical devices, which have been the subject of multiple studies [64, 68]. Our objective is to output a table that automatically structures the attributes that are distributed in the ~20-page **PDFs**, for instance the `device classification`, `predicate device code`, and `indications for use`.

Systems designed to tackle this problem must balance a three-way tradeoff between **cost** (data lakes may hold millions of documents), **quality** (output tables should be able to accurately support an analyst's queries), and **generality** (different data lakes have different document types and structure). See Section 2 for a formal task definition and further discussion of this tradeoff.

Given the range of formats, attributes, and domains across documents, prior systems rely on simplifying assumptions (e.g. handling one document format). The majority of works focus on structuring HTML [12, 15, 25], assuming the attributes and values are at specific positions in the HTML-DOM [22, 43, 44, 67]. For unstructured text, current approaches use linguistic tools (e.g., dependency parsers) to introduce structure [15, 25, 31, 48] and then apply heuristic rules over the resulting structure to extract information. The documents in Example 1 highlight the limitations of the prior approaches: they lack (e.g. HTML) structure and, consistent with recent evaluation efforts [67], we find the SoTA approaches for unstructured text perform poorly on long semi-structured PDFs (See [5]). Some systems assume there is a human-in-the-loop, labeling data and writing heuristic rules for extraction [53, 57], while others assume access to annotated training documents from the domain [22, 43, 44]. Researchers manually annotated the reports in Example 1 [64].

* Equal contribution.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 2 % ISSN 2150-8097.
doi:10.14778/3626292.3626294

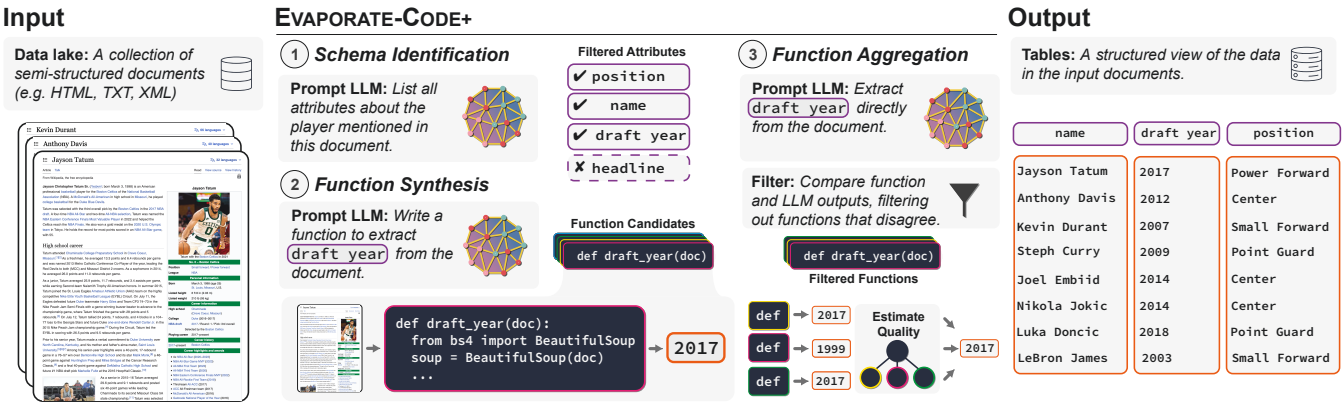


Figure 1: The user provides a collection of documents (e.g. NBA player bios) and EVAPORATE outputs a table by identifying attributes and populating columns. EVAPORATE avoids running expensive LLM inference on all documents by (1) synthesizing the key attributes from a small sample of documents and (2) synthesizing (e.g. Pythonic) functions that then are reused at scale to process documents. Because function quality is variable, EVAPORATE (3) applies an algorithm that generates many candidate functions and ensembles their extractions using weak supervision.

In this work, we explore whether we can improve generality by leveraging *large language models* (LLMs). An LLM is a deep learning model that is pretrained on broad data and can be adapted to diverse tasks, from machine translation to data wrangling [14, 46]. At inference time, the models take as input a natural language task description termed a *prompt* [11, 14] and generate a natural language response. See Section 2.3 for more background on LLMs.

EVAPORATE. (Section 3) We present EVAPORATE, a system that uses LLMs to produce structured views of semi-structured data lakes. Our evaluation spans 16 real-world settings from movie and university websites to *FDA 510(k)* reviews [22, 32, 34, 37, 43, 64, 68].

The user inputs a collection of documents and EVAPORATE automatically identifies the schema and performs extraction to populate the table. Our implementation requires *no customization, training, or human effort* to support the diverse evaluation settings. We propose two fundamental strategies for implementing this interface, identifying a tradeoff between their cost and quality:

(1) EVAPORATE-DIRECT (Figure 2) The LLM directly extracts values from documents.
(2) EVAPORATE-CODE (Figure 4) The LLM *synthesizes code* that is then applied to process documents at scale.

EVAPORATE-CODE is cheap, but underperforms EVAPORATE-DIRECT by 24.9% (13.8 F1 points) averaged across our evaluation settings. We thus seek a new code synthesis approach. We present EVAPORATE-CODE+, which achieves better quality than direct extraction. Our insight is to synthesize many code snippets for extraction and ensemble their outputs using weak supervision.

Direct Extraction (Section 3.1). Our first implementation, EVAPORATE-DIRECT, applies a *single prompt* (included in [5]) to each document in the input. The prompt instructs the LLM to both identify the schema and extract values. Remarkably, we find that in some settings, with a single prompt and no task specific modifications, performance is already competitive with state-of-the-art systems that rely on domain specific assumptions and training.

However, this implementation is very expensive. LLMs are optimized for interactive, human-in-the-loop applications (*e.g.* ChatGPT) [65], not high-throughput data processing tasks [56]. The number of tokens processed by an LLM in EVAPORATE-DIRECT grows *linearly* with the size of the data lake. As of March 2023, applying OpenAI's models to the 55 million Wikipedia articles would cost over $110k (`gpt-3.5`, $0.002/1k tokens) and $1.1M (`text-davinci-003`, $0.02/1k tokens) dollars [1, 49]. There are *billions* of webpages on the broader Internet [35] and the facts change over time. For instance, NBA players are added to Wikipedia, a player's `team` changes after trades, and the `points per game` metric changes after every game. Data processing is a *routine expense* (repeated by multiple data analysts), not a one-time cost [55].

Code Synthesis (Section 3.2). *Can we produce the structured table using a sublinear pass of the LLM over the documents?* We propose EVAPORATE-CODE, which splits the task into two sub-tasks: (1) identify the table schema and (2) extract values. This view allows us to exploit the distinct *redundancies* of each sub-task that occur when running LLM inference on every document:

(1) *Schema Generation.* In order to identify a schema, we only process a small sample of documents with the LLM. This succeeds because there is redundancy in the attributes mentioned across documents. For e.g., in Example 1, most reports mention a `predicate device name`.
(2) *Function Synthesis.* We prompt the LLM to synthesize (*e.g.* Pythonic) *functions*, that can be applied at scale across the documents. This works because of redundancy in the formatting of attribute-value pairs. For e.g., the FDA 510(k)s use the consistent format "`Predicate device name: k`".

The number of tokens processed by the LLM in EVAPORATE-CODE is *fixed* and does not grow with the size of the data lake (as illustrated in Figure 3), addressing the cost issues of EVAPORATE-DIRECT. However, the LLM synthesizes variable quality information extraction functions. The extractions are up to 14 points worse in Pair F1 score than those produced using EVAPORATE-DIRECT.

Code Synthesis + Aggregation. (Section 3.3) To improve quality while keeping costs low, we propose Evaporate-Code+. Studying the synthesized functions, we observe some only work for a narrow slice of documents, while others exhibit syntactic and logical errors. To reduce variance, we synthesize many candidate functions, then estimate their quality and aggregate their extractions using *weak supervision*. This builds on our work [4], which broadly applies weak supervision to prompting for the first time.

Weak supervision (WS) is a statistical framework for modeling and combining noisy sources with varied coverages without any labeled data [53, 62]. However, WS is typically applied over *human-generated* functions while our setting consists of *machine-generated* functions. This presents issues when attempting to apply existing WS tools. (1) WS theoretically assumes all noisy sources are better than random performance (50% accuracy), yet 40% of our generated functions are *below 25%* (Section 3.2). (2) WS attempts to deploy functions that achieve high quality on narrow slices of data (high precision), and allow the function to *abstain* on data external to the slice (low recall). While humans can express when functions should abstain, the machine-generated functions do not contain this logic. To handle the *open* WS setting, we introduce a new algorithm for ensembling the functions (Algorithm 1).

We summarize our overall contributions as follows.

(1) **Our system offers new capabilities for the long-studied structured view generation problem.** Existing systems require in-domain training and handle limited document formats (e.g. HTML [16, 22, 24, 43]). Evaporate requires no training and succeeds on different document formats (HTML, PDF, TXT) off-the-shelf. (Section 4).
(2) **We study a new tradeoff space between direct extraction and code synthesis for data tasks.** Evaporate *asymptotically* reduces the number of tokens that the LLM needs to process to generate the outputs. At 10k documents per evaluation setting, this amounts to a 110x cost reduction. Further, prior works using LLMs for data tasks require users to manually write prompts [46]. Evaporate is built with task-agnostic prompts that generalize across settings.
(3) **We present an algorithm and theoretical analysis for applying weak supervision to open-ended functions and extraction tasks.** Although Evaporate-Code is more efficient, Evaporate-Direct achieves significantly higher quality. Using our algorithm, Evaporate-Code+ outperforms Evaporate-Direct, which directly processes every document, by 10.1 F1 points (18%) (Table 3).
(4) **We extensively validate the system on 16 data settings from 5 domains and 3 data formats, and across 4 LLMs.** (1) Evaporate outperforms the SoTA *learned* baseline systems by 3.2 F1 points (6%) when generating tables (both schema generation and extraction) end-to-end, and 6.7 F1 points (10%) on the extraction step. (2) Evaporate-Code+ achieves a 10.1 F1 point increase over Evaporate-Direct, using text-davinci-003. (3) Across four unique LLMs we show the relative quality of Evaporate-Direct vs. Evaporate-Code+ remains consistent.

We define the problem in Section 2. We present Evaporate in Section 3, evaluations in Section 4, and related works in Section 5.

2 PRELIMINARIES

We first define the problem setting and system desiderata.

2.1 Problem Setting

We study the problem of constructing a structured view (*i.e.* database table) of a set of semi-structured documents (*e.g.* HTML, PDF, TXT). Formally, we define the problem as follows:

- **Input:** User provides a set of n semi-structured documents $D = \{d_1, d_2, ...d_n\}$ (*e.g.* A collection of FDA 510(k) reviews for premarket notification submission for medical devices).
- **Output:** System outputs a table defined by a set of attribute names $A = \{a_1, a_2, ...a_m\}$ (*e.g.* a_1 =`indications for use`, a_2=`classification`) and a set of n extracted records for $R = \{r_1, r_2, ...r_n\}$, one per document, where r_i is an m-tuple (*e.g.* r_1 = ("fracture", "x-ray")).

Unlike prior work which proposes systems that rely on manual labeling [57] or manual prompt tuning [46, 59], we aim to develop *automated* solutions, which require no user effort.

Measuring System Quality We compare the generated table (A, R) to a manually curated "ground-truth" table $(\hat{A}, \hat{R})$. The coverage of an attribute refers to the fraction of documents that include the attribute and its value. Following prior work, we prioritize attributes with high *coverage*, which tend to be useful for analysis [16, 18]. We measure agreement between the tables using Pair F1. For additional details on our evaluation setup, see Section 4.3.

2.2 System Desiderata

Current systems for producing structured views are limited in their generality, cost/flexibility, and quality/usability [16, 18, 48, 67]. Here we review the existing systems.

Generality. *The ideal system will generalize across document formats and domains, without manually engineered rules or task-specific training.* This is important because the input documents D could focus on any imaginable topic or use any file format [67]. Existing systems featurize documents by tagging the named entities (NER), dependency parse tree, and part-of-speech (POS), and train a model to predict whether a span of text is a useful fact [39]. Unfortunately, the performance of the parse, NER, and POS tags drastically degrade on semi-structured data (e.g. HTML elements) and longer sequences of text (i.e. full documents) [67]. We provide detailed error analysis [5]. A specialized class of systems focuses on processing semi-structured web HTML documents by leveraging the HTML DOM tree as features [13, 16, 22, 25, 44, inter alia.]. However, the systems thus do not support other document formats.

Cost. *The ideal system will enable users to manage a cost-coverage tradeoff, rather than requiring them to extract "all-or-nothing".* The existing systems are built to extract *all* possible facts in the documents, without prioritizing important attributes or allowing the user to influence what is extracted [21, 67]. Processing every line of every document can be expensive. To mitigate this, the user can define the attributes of interest then apply a closed IE system for extraction, however this requires upfront human effort. **Desiderata**: The ideal system will enable users to manage a cost-coverage tradeoff, rather than requiring them extract "all-or-nothing".



ReCG: Bottom-Up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework

Joohyung Yun
POSTECH
Pohang, Republic of Korea
jhyun@dblab.postch.ac.kr

Byungchul Tak*
Kyungpook National University
Daegu, Republic of Korea
bctak@knu.ac.kr

Wook-Shin Han*
Graduate School of AI
POSTECH, Republic of Korea
wshan@dblab.postech.ac.kr

ABSTRACT

The schemalessness, one of the major advantages of JSON representation format, comes with high penalties in querying and operations by denying various critical functions such as query optimizations, indexing, or data verification. There have been continuous efforts to develop an accurate JSON schema discovery algorithm from a bag of JSON documents. Unfortunately, existing schema discovery techniques, being top-down algorithms, face challenges from the lack of visibility into children nodes of JSON tree. With absence of the information about lower-level JSON elements, top-down algorithms need to employ assumptions and heuristics to decide the schema type of nodes. However, such static decisions are often violated in datasets which causes top-down algorithms to perform poorly. To overcome this, we propose an algorithm, called ReCG, that processes JSON documents in a bottom-up manner. It builds up schemas from leaf elements upward in the JSON document tree and, thus, can make more informed decisions of the schema node types. In addition, we adopt MDL (Minimum Description Length) principles systematically while building up the schemas to choose among candidate schemas the most concise yet accurate one with well-balanced generality. Evaluations show that our technique improves the recall and precision of found schemas by as high as 47%, resulting in 46% better F1 score while also performing 2.11× faster on average against the state-of-the-art.

PVLDB Reference Format:
Joohyung Yun, Byungchul Tak, and Wook-Shin Han. ReCG: Bottom-Up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework. PVLDB, 17(11): 3538 - 3550, 2024.
doi:10.14778/3681954.3682019

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/postechdblab/VLDB2024_ReCG.

1 INTRODUCTION

JSON (JavaScript Object Notation) is a widely used data representation format that has become de-facto standard for RESTful Web API's data exchanges [2, 23, 30, 44] and big data analytics [5, 10, 16, 27, 31, 40, 41]. Accordingly, efficient processing of massive volumes of JSON data and querying has become critical for services and operations. However, the efficiency of these operations is often hindered by the lack of schemas for the JSON documents at hand. Although the lack of strict schema enforcement of JSON format offers flexibility and rapid deployability, it instead exacerbates the data management costs by making it difficult to apply various operations [42]. Schemas are essential for many important use cases such as JSON validation [3, 34], JSON parsing acceleration [27, 31], migration of two data sources with different schemas [43], query formulation [15, 48], query optimizations [7, 9, 18, 33], fine-grained access control [42], JSON-to-relational data translation [13, 25, 46], and query answering [47]. Despite these practical benefits, JSON schemas are often nonexistent or unavailable to the users and, thus, have to be derived from a given set of JSON documents.

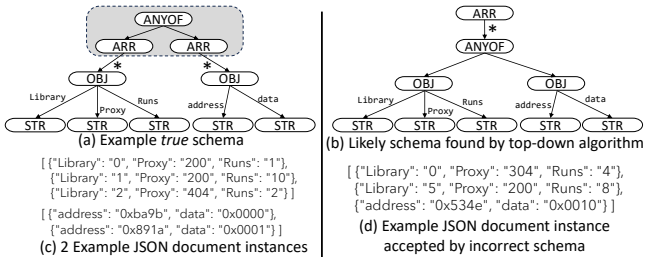


Figure 1: Illustration of top-down approach's limitation. It may lead to incorrect results (b) due to lack of visibility.

Due to the importance of JSON schemas, the JSON schema discovery (JSD) problem and topics related to the JSON schema formalization have been investigated [3, 4, 6, 11, 16, 21, 34, 38, 42]. Despite many efforts, deriving correct JSON schemas from JSON documents remains difficult due to several reasons. First, the ground truth schema set we want to derive may not exist in the first place for the given JSON documents. It is not uncommon that JSON documents are created without explicit schemas defined in advance. Users of JSON data prefer to create, load, and use the data quickly without the burden and delay from the schema construction task [32, 39]. The lack of a predefined schema also implies that the representation of objects in JSON can be inconsistent and has a high degree of variability [24]. Second, the derivation of a huge number of *correct* and *valid* schema sets is possible for a given JSON document set, but with varying degrees of generality. If the derived schemas are too general, the data validation becomes ineffective [38]. Moreover, the specific details of the schemas have to be eventually encoded into the query which increases the query processing overhead during the execution. On the other hand, if too specific, the size of the schema set increases and the schema handling costs rise significantly. A working solution to JSD problem should be able to find the schema with the right balance between representational simplicity and details within the vast search space of valid schemas.

*Corresponding authors
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 17, No. 11 ISSN 2150-8097.
doi:10.14778/3681954.3682019

A JSON schema can be viewed intuitively as having a tree-like hierarchical structure as shown in Figure 1 (a). The intermediate nodes in this tree structure are either objects or arrays that recursively hold other objects, arrays, or primitive types. The goal of the JSON schema discovery (JSD) problem is to learn this template structure from a given bag of JSON documents. Two example JSON documents are shown in Figure 1 (c). The top-down style of JSON schema discovery, adopted by the state-of-the-art algorithms [4, 6, 38], starts processing from the root and it proceeds down to the subsequent child levels repeatedly. However, this top-down approach faces the following nontrivial challenges. When a new node is encountered, it can be difficult to determine the correct node types (e.g., `ARR`, `OBJ`, `ANYOF`) since it has to be done without the knowledge of yet unseen lower-level nodes. This inevitably leads to the use of heuristics to make the best-effort decision of what the true type of the current node is. For example, at the shaded region of Figure 1 (a), it illustrates one possible case where the top-down algorithm failed to differentiate two types of objects and, thus, comes up with the incorrect (and overly general) schema as shown in (b) which accepts a mixture of objects. In addition, existing algorithms make a decision of node types rigidly even when there are other possible candidates at each decision point. This leaves little chance to rectify incorrect decisions made at the upper level when the processing reaches the descendant nodes where decision errors become evident.

To overcome the limitations of top-down approaches, we have developed a methodology, called `ReCG`, that employs three techniques. *First, we systematically explore candidate JSON schema sets comprehensively by incrementally generalizing them from specific schemas to the most general ones.* As Figure 2 illustrates, there can be schema sets with varying generality for the same JSON document sets at some point during the JSON schema discovery. We do not attempt to guess the correct one during processing as any choice has the potential to grow to be the true schema when schema discovery is completed. *Second, we employ the MDL (Minimum Description Length) metric to our JSON schema discovery problem as a metric that guides us to select the most promising candidate schema set during the navigation of the search space.* The MDL metric, used successfully in `XTRACT` [22] for computing DTD's information size, can be used here to compute the number of bits required to represent the schema and the data (JSON document) instances. We demonstrate in our evaluation its validity as a metric for assessing the goodness of JSON schema set. *Third, we design a novel bottom-up style JSON schema discovery algorithm* that departs from the existing approaches. We argue that the bottom-up style of processing is inherently better suited to the JSON schema discovery problem. At any intermediate node, the determination of a node type is better informed since the knowledge of descendant nodes is already acquired. Any two nodes of identical type can actually be disparate when their children and descendants are taken into account. By using this bottom-up style approach, we can expect to reduce the uncertainty in making decisions and the reliance on heuristics.

We have designed and implemented the prototype of our technique and evaluated it using 20 real-world JSON datasets. Our evaluation showed that `ReCG` delivered the 42.57 ~ 47.80% improved F1 score compared to the state-of-the-art technique. We observed this was due to 39.36 ~ 46.84% improvements in recall and 15.55 ~ 23.33% improvements in precision. In terms of the algorithm running time, `ReCG` outperformed the state-of-the-art by 2.11×.

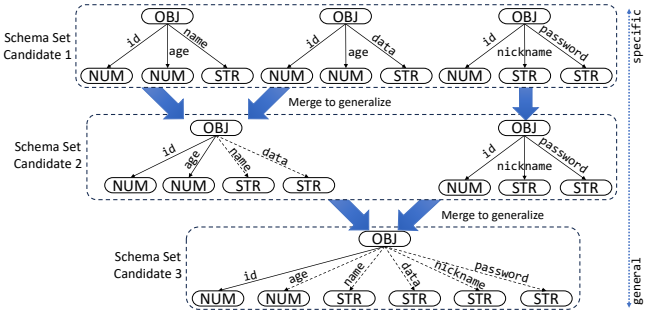


Figure 2: Schema sets with varying generality. More general schema sets are created by merging more specific sets.

J	::=	$P \mid O \mid A$	
P	::=	`null` | `true` | `false` | $n \mid s$	($n \in$ **Number**, $s \in$ **String**)
O	::=	$\{\ k_1 : J_1, \ldots, k_n : J_n\ \}$	($n \geq 0, i \neq j \implies k_i \neq k_j$)
A	::=	$[\ J_1, \ldots, J_n\ ]$	($n \geq 0$)

Figure 3: Grammar of JSON documents

We make the following contributions. First, we design a novel bottom-up algorithm, called `ReCG` for the JSON schema discovery problem that addresses major issues of existing algorithms. Accompanied by a technique for systematic search of schema sets by the specificity and the metric for assessing the goodness, our technique exhibits much-improved precision and recall against competing algorithms. Second, we provide supporting evidence that demonstrates the effectiveness of our technique using real-world data sets. Third, we make a prototype implementation of our technique as well as the dataset publicly available for the research community.

2 BACKGROUND

2.1 Definition

2.1.1 JSON document. We adopt the definition of JSON document introduced in the work by Baazizi et al. [6]. JSON documents are a set of strings that are derived from the grammar described in Figure 3. The derivation starts from a non-terminal J. It transitions to either P (primitive JSON document), O (object), or A (array). The non-terminal P further transitions to one of the following:

- *Number* type JSON document: C-like number. The set of these numbers is denoted as **Number**.
- *String* type JSON document: double-quoted sequence of zero or more Unicode characters. The set of such double-quoted sequences is denoted as **String**.
- *Boolean* type JSON document: `true` or `false`
- *Null* type JSON document: `null`

The non-terminal O is derived to an object, which is an unordered set of (key, JSON document) pairs. An object begins with '{' and ends with '}', and the pairs are separated by ','. We denote the pairs within an object as *key-value pairs*. A is derived to an array, which is an ordered sequence of JSON documents. We call the JSON documents within the sequence of array as *elements*. An array begins with '[' and ']', with elements separated by ','. The JSON document can be recursively defined. That is, JSON documents derived from J can recursively occur within objects and arrays.

2.1.2 JSON Schema. *JSON schema*s are a set of strings in the Backus-Naur form grammar, introduced by Pezoa et al. [34]. We adopt a subset of Pezoa et al.'s production rules since real-life schemas use only a limited subset of grammars, as introduced specifically by Spoth et al. [38]. The grammar of our interest is shown in Figure 4, and we denote a JSON schema as $\mathcal{S}$. In this grammar, non-terminals are expressed in *italic*, and the terminals are expressed in consolas.

2.1.3 Homogeneity and Heterogeneity of Object and Array Schemas. For object schemas, derivations from *homO* and *hetO* impose constraints on the key-value pairs of an object. Derivation from *homO* imposes objects to be *homogeneous*; keys that can be present within an object's key-value pairs are determined as *key*s. Also, the corresponding value for each *key* must be validated against the schema derived by *JS* (look $key : JS$ at the right-hand side of *prop*). *req* is used to list the keys that must be present in an object. Meanwhile, derivation from *hetO* imposes objects to be *heterogeneous*. The number of key-value pairs is unconstrained and the keys may be any random string. However, the values must be validated against the schema derived from *JS*. We name object schemas that have *homO* non-terminal derived as *homogeneous object schema*s, and *hetO* derived as *heterogeneous object schema*s. Object schemas with derivations from both *homO* and *hetO* are *composite object schema*s.

For array schemas, derivation from *homA* imposes arrays to be *homogeneous*; the number of elements within an array must be fixed, and each element at index i must be validated against the ith schema paired by the key "items". Whereas, derivation from *hetA* imposes arrays to be *heterogeneous*. The number of elements may vary for an array, but its elements should all be validated against the schema derived from the non-terminal *JS* paired by the key "items". We name array schemas that have *homA* non-terminal derived as *homogeneous array schema*s, and *hetA* derived as *heterogeneous array schemas*.

Lastly, derivation from *anyOf* is named as *anyOf* schema. It can have many schemas derived from *JS*s paired by the keyword "anyOf". An anyOf schema validates against a JSON document j if and only if one or more of its derivations can validate against j.

2.2 JSON Document Instance and Schema Representation

2.2.1 JSON Instance Tree. We model JSON documents as *JSON instance tree*s, which are node-typed, node-labeled and edge-labeled trees. Instance trees are extensions from *JSON tree*s introduced by Hutter and Augsten et al. [16]. We define an instance tree as $I = (V_I, E_I, \Psi_I, \Lambda_I, \Gamma_I)$ with the following notations.

- V_I is a set of nodes.
- $E_I \subseteq V_I \times V_I$ is a set of edges.
- $\Psi_I : V_I \rightarrow \{obj, arr, prm\}$ maps each node to its node type.
- $\Lambda_I : V_I \rightarrow$ **Number** $\cup$ **String** $\cup\{$true, false, null, $\epsilon\}$ maps a node to its label. ϵ means an empty label.
- $\Gamma_I : E_I \rightarrow$**String** $\cup\{\epsilon\}$ maps each edge to its edge label.

We transform a JSON document to a JSON tree as follows. Primitive JSON documents each become nodes with the type of *prm*. The nodes are further labeled with the values of JSON documents. Objects become nodes of type *obj* (labeled as ϵ) with the nodes of its values as children. The edge connecting an object node to its child is labeled with the key corresponding to the value of the child. An array is converted to an *arr* (labeled as ϵ) node with the i-th element becoming the i-th child subtree.

JS	::= { *SchCont* }	*prop*	::= , "properties": { (*key* : *JS*,)* }
SchCont	::= *strSch* | *numSch* | *boolSch* | *nullSch* | *objSch* | *arrSch* | *anyOf*	*req*	::= , "required": [(*key* ,)*]
		key	::= *s* (*s* ∈ **String**)
strSch	::= "type": "string"	*hetO*	::= , "additionalProperties": *JS*
numSch	::= "type": "number"	*arrSch*	::= "type": "array" (*homA* | *hetA*)
boolSch	::= "type": "boolean"	*homA*	::= , "items": *JS*
nullSch	::= "type": "null"	*hetA*	::= , "items": [(*JS*,)*]
objSch	::= "type": "object" *homO*? *hetO*?	*anyOf*	::= "anyOf": [(*JS*,)+]
homO	::= *prop req*?		

Figure 4: BNF grammar of JSON schemas.

2.2.2 Schema Tree. We also model a JSON schema as a tree, as it is expressed in recursive forms that can be captured well as a tree. A schema tree is a node-labeled, edge-labeled, and edge-typed tree defined as $s = (V_S, E_S, \Lambda_S, \Gamma_S, \Phi_S)$ with the following definition.

- V_S is a set of nodes. Each node represents a JSON schema.
- $E_S \subseteq V_S \times V_S$ is a set of edges.
- $\Lambda_S : V_S \rightarrow \{$ NUM, STR, BOOL, NULL, OBJ, ARR, ANYOF $\}$ maps each node to its node label which corresponds to the type of the schema.
- $\Gamma_S : E_S \rightarrow$ **String** $\cup\{ *, \epsilon \}$ maps each edge to its edge label. An edge label expresses either the derivation from *key* part in the right side of *prop*, or an "additionalProperties" of *hetO* part with a Kleene star $*$. Edge label $*$ indicates the heterogeneity of the schema node being the source node of that edge, while *key* indicates homogeneity.
- $\Phi_S : E_S \rightarrow \{$ Required, Optional, $\epsilon \}$ maps each edge to its type. The type of edge is only meaningful when $\Lambda_S(v)$ of its source node v is OBJ. It indicates whether a key of a homogeneous object schema is Required, or Optional. In our visualization, edges of Required type are expressed as solid lines, and edges of Optional are expressed as dotted lines.

For both instance trees and schema trees, we define an operator $v[l]$ for a node v and a label l.

$$v[l] = \begin{cases} v' & \text{if } \exists l \in \textbf{String},\ e = (v, v'),\ \Gamma(e) = l \\ v' & \text{else if } \exists l \in \mathbb{Z}^+,\ e = (v, v'),\ v' \text{ is } l^{th} \text{ child of } v \\ \epsilon & \text{otherwise} \end{cases} \quad (1)$$

It is an operator that returns v', the child subtree of v such that an edge $e = (v, v')$ exists which $\Gamma(e)$ is l, or the lth child subtree of v if $\Gamma(e)$ are all null. If such an edge does not exist, the return value is ϵ. We denote e_l as an edge of which $\Gamma(e) = l$. We denote $edgelabels(v)$ as a set of labels of v's outgoing edges.

2.3 Schema Assessment Metric: MDL Cost

The ideal set of JSON schemas should be general enough to accept all given positive JSON document instances and be specific enough to reject all negative JSON document instances. We need a method to quantify such quality of a set of JSON schemas to navigate through the search space. For this, we adopt the Minimum Description Length (MDL) principle [22, 35, 36] to our JSD problem.

The MDL principle, based on the information theory, offers a way to evaluate the adequacy of models inferred from a data set. It states that the best theory to infer a model from a set of data is the one that minimizes the sum of two components — *Schema Representation Cost* (SRC) and *Data Representation Cost* (DRC). The best model (i.e., set of JSON schemas in our problem settings) inferred